

β

This project is currently under beta-test, until it is validated by a fair number of students in the 42 community. Please report any typo, incoherence, inconsistency, error, using the form <https://tally.so/r/3lVDJo>





Inception-of-Context (IoC)

Summary: This document is a System Administration and Artificial Intelligence-related exercise.

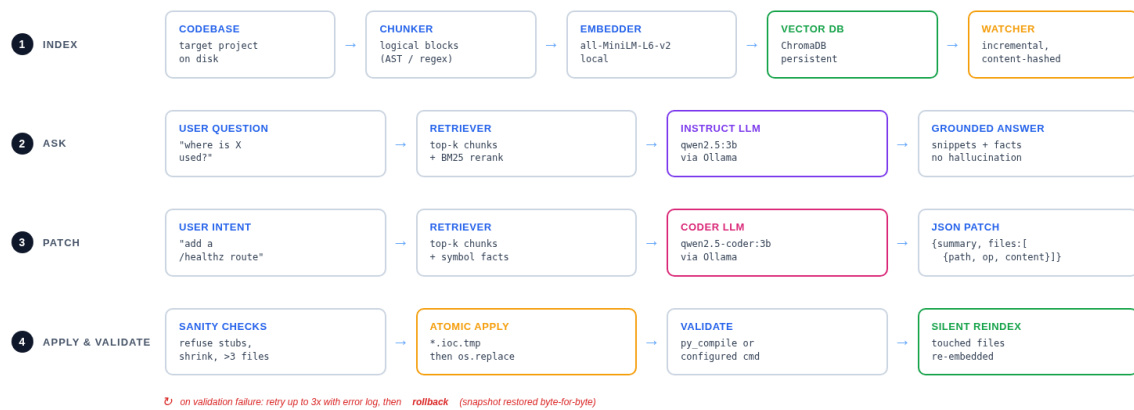
Version: 1.0

Contents

I	Preamble	2
II	Foreword	3
III	Introduction	4
IV	General guidelines	5
V	Demo application	6
VI	Mandatory part	7
VI.1	Part 1: Indexing and synchronization	7
VI.2	Part 2: Architect API and RAG	9
VI.3	Part 3: Generation, application and validation loop	10
VII	Bonus part	13
VIII	Submission and peer-evaluation	14

Chapter I

Preamble



The three parts you will build: index the codebase (Part 1), answer questions about it with a small instruct model (Part 2), and generate-apply-validate patches with a coder model that rolls back on failure (Part 3).

Chapter II

Foreword

	PAY A VENDOR \$20 / MONTH SO THEIR LLM CAN READ YOUR CODEBASE.
	SPEND A WEEKEND BUILDING YOUR OWN, AND NEVER SHIP A TOKEN AGAIN.
Inception-of-Context	

The software industry is in the middle of a quiet shift. The way new code gets written, reviewed, and shipped is being rebuilt around language models. The tooling has a name: **AI-Native Engineering**. The assistants that read a codebase, retrieve the right context, draft a patch, and validate it before a human reads the diff are no longer a research demo. They are how senior teams ship.

Most engineers use one. Almost nobody has built one.

This project is your chance to be on the other side of the API. A vector index over the code, a retriever that answers questions in real time, a small local language model that turns your intent into a patch, and an apply-and-validate loop that refuses to leave the project broken. Four pieces that talk to each other, and a hard rule: if the patch breaks the project, the system rolls back as if nothing happened.

Chapter III

Introduction

This project is a hands-on introduction to the three core ideas of AI-Native Engineering: RAG (Retrieval-Augmented Generation) applied to source code, orchestration of a local language model, and automated code modification under validation. You will work your way up from a passive index of the codebase to an autonomous patch loop that proposes, applies, and self-corrects.

Everything must run **locally**. Any call to a remote LLM service is forbidden and results in a grade of zero. The point of the project is precisely to prove that the whole stack fits on a laptop, with no vendor lock-in and no token bill.

The three parts that follow share one shape: the system observes a real codebase, reasons about it with a small local model, and only commits to a change once it has been validated. Nothing works on a single piece in isolation; the index of Part 1 feeds the retrieval of Part 2, and the API of Part 2 carries the patch loop of Part 3. Read the whole document before writing any code.



If you have done *Inception-of-Things*, you already know the recursive flavor of these projects: a system that observes its own state and acts on it. If you have not, we recommend doing the two in parallel. They live on opposite sides of the same idea: one rebuilds your infrastructure from a Git push, the other rebuilds your code from a textual intent.

Chapter IV

General guidelines

- The whole project has to be done in an **isolated environment**. A virtual machine or a container works. Docker is the easy choice.
- You have to put all the configuration and source files of your project at the root of your repository, in the folders **p1**, **p2**, **p3**, and optionally **bonus**.
- Pick any language for the orchestrator, the indexer and the frontend; one your peer can read and run is what matters. All code snippets, file names, hints and default commands in this document are written in **Python** for consistency, but every one of them is an example. The same deliverables in another language, with equivalent tooling, are equally valid.
- The language model has to run **locally**, through **Ollama** or an equivalent.
- The vector database has to run locally too. For this subject, **ChromaDB** in **PersistentClient** mode is the reference.



You can use any local model in the 3-billion-parameter class or smaller. `qwen2.5-coder:3b` (about 1.9 GB on disk) is a comfortable sweet spot for this subject.



One model is enough to pass. Two often do better. An instruct model carries the conversational Q&A in Part 2; a coder model carries the patch JSON in Part 3. Both fit the 3B budget. `qwen2.5:3b` for /ask and `qwen2.5-coder:3b` for the patch loop is a natural pairing.

Chapter V

Demo application

Your repository ships a small demo application of its own, used to drive the indexer, the retriever and the patch loop in front of a peer. Wherever the rest of this document refers to the *target project* or the *target directory*, it means this demo application: the indexer points at it, the retriever pulls chunks out of it, and the patch loop writes new code into it. Its language, structure and content are your choice. Its quality is graded only indirectly: a small app that exercises every mandatory feature is worth more than a large one the system cannot index in time for the defense.

Start small. A few files, one class, a handful of methods is enough. The indexer needs something to chunk, the retriever needs something to find, and the patch loop needs a place to land a new function. Three or four cooperating files clear that bar without filling your laptop with embeddings you do not need.

Once the mandatory part works end to end, you can grow the demo app: add a second class, add a few cross-file imports, plug a richer validation command into `ioc.config.yml`. Growing the demo is how you stress-test your own system, not the goal of the project.



Do not point the system at a large open-source codebase, the Linux kernel, or another sprawling project you happen to have on disk. A 3B local model has a few thousand tokens of context to play with. A fifty-thousand-line codebase will not retrieve well, will not produce clean patches, and will not validate inside three iterations. Start with something you can read in two minutes.



A handful of files with one class carrying four or five methods, plus an entry point, is enough to exercise the logical chunker, cross-file retrieval and a non-trivial validation hook. The validation command in `ioc.config.yml` is yours to configure: a compile check, a unit-test suite, a linter, or a chain of any of these.

Chapter VI

Mandatory part

The three parts below must be done in order.

VI.1 Part 1: Indexing and synchronization

Everything starts with the index: a vector database that mirrors a target project and stays in sync with it on disk.

The indexer walks a target directory, splits each source file into logical blocks, embeds them, and persists everything to local storage. A reference layout is a `p1/index.py` script that takes the target path as an argument and produces a persisted **ChromaDB** collection. The same indexer then ships a watch mode that reacts to filesystem events and keeps the collection up to date.

A few hard constraints:

- Chunking follows the code's logical structure (one chunk per function, class, or configuration block). A fixed-size chunker does not pass this part.
- Embeddings are computed locally with a lightweight model such as **all-MiniLM-L6-v2**. The model is downloaded once, after which the indexer must work fully offline.
- Reindex is incremental. Modifying a single file touches only that file's chunks, and deleting a file removes its chunks from the database.
- The indexer skips `node_modules`, `.git`, `dist`, `build`, `venv`, `.venv`, `__pycache__`, hidden directories, and binary files. In particular, the persisted vector store must never be indexed or watched: either keep it outside the target directory, or exclude its folder explicitly. Otherwise the watcher reacts to its own writes.

By the end, the index is already useful to a human reader, so the dashboard's first page lives here too. The Overview tab confirms at a glance that the index reflects reality: indexed chunk count, target path, embedding and LLM model in use, per-file chunk count, and a live activity feed that fires whenever the watcher picks up a change. No Q&A and no generation yet; that is Part 2.

Inception-of-Context (IoC)

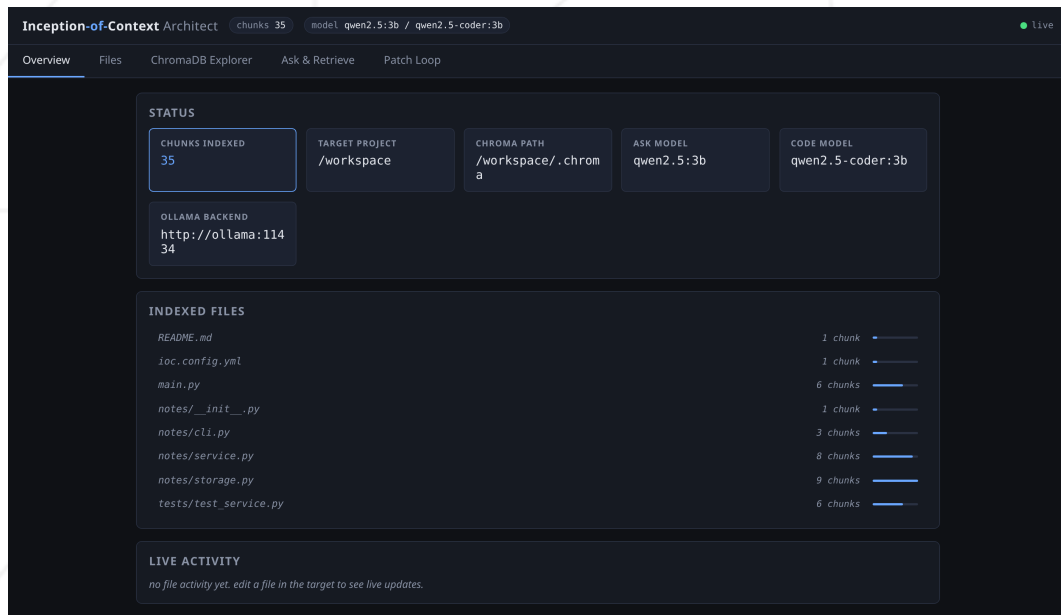


Figure VI.1: *

Overview tab after a fresh run on a small target. The status bar reflects what the indexer just persisted; the file list reflects the on-disk reality.



A full reindex on every modification does not pass this part. The watcher must be incremental and must survive deletions.



Hash each chunk so you can detect actual changes. On some filesystems, in particular Docker bind mounts on top of btrfs or overlay, inotify silently drops events. Have a polling fallback ready.



Python's ast module hands you the exact line range of every function, async function and class, decorators included. Walking the tree catches methods nested inside a class, which a regex anchored on ^def silently misses. The regex path is still useful as a fallback for non-Python files and for a Python source that fails to parse.

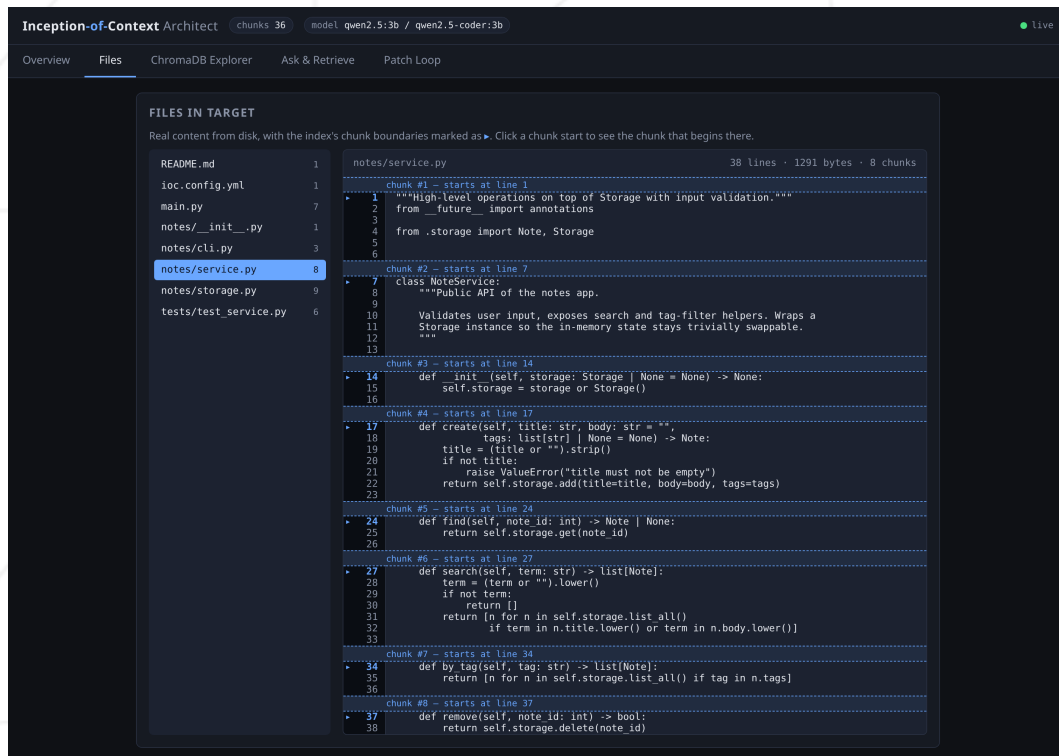


Figure VI.2: *

Files tab on a target with a real class. Each chunk boundary (the ► marker in the gutter) lines up with a method definition, not with an arbitrary line count.

VI.2 Part 2: Architect API and RAG

With the index built, you can query it. An HTTP server (**FastAPI** is a sensible default on the reference stack) exposes the collection and adds two new capabilities on top of it: retrieval (return the most relevant chunks for a given intent) and retrieval-augmented generation (retrieve those chunks, feed them to a local LLM, and return a grounded answer).

The server lives next to the watcher and shares its **ChromaDB** collection through a single client per process. The local model runs through **Ollama** or an equivalent. Any 3B local model works here; an instruct model such as **qwen2.5:3b** is a natural fit for the conversational Q&A, while a coder model is better kept for the patch loop in Part 3. A single model for both parts is still acceptable.

How you carve up the HTTP surface is up to you, but a peer needs concrete handles to grade your work. A reasonable baseline exposes the index state (**GET /status**, **GET /files**, **GET /file?path=...**, **GET /chunks** with pagination), a retrieval endpoint (**POST /context** returning the top-k chunks with their similarity score), a Q&A endpoint (**POST /ask** for retrieve-then-generate), and a file events stream (**GET /events** over Server-Sent Events).

The dashboard grows a new tab, *Ask & Retrieve*, where the user types an intent, picks a **k** (the number of chunks to retrieve, surfaced on the form), and either runs retrieval

alone or hands the result to the model.

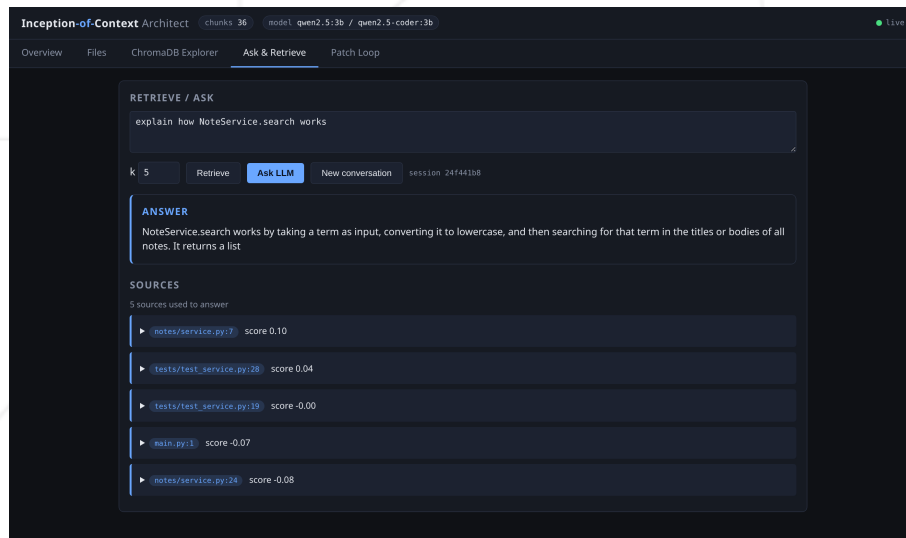


Figure VI.3: *

Ask & Retrieve tab. Above, the question and the top- k selector. Below, the model's grounded answer and the chunks that fed it, ranked by similarity.

Two question shapes are particularly easy for a small model to get wrong: "is there a function called X?" and "what functions exist in this file?". On those, /ask must not hallucinate symbols that are not in the index. The whole /ask endpoint is graded on staying truthful, not on sounding clever.



If you cannot get clean answers on those, do not try to fix it inside the prompt. Think about what your own index already knows for sure, and how you could pre-resolve that knowledge before the model is allowed to speak. There is more than one way to thread that needle.



Embedding similarity is strong on paraphrased intent and weaker on exact identifier matches. A query like "where is authenticate used?" can miss the right chunks entirely. A small in-memory BM25 index over the Chroma documents (rank_bm25 is enough) re-ranks well and stays cheap. Keep ChromaDB as the canonical store; BM25 is a re-ranker, not a replacement.

VI.3 Part 3: Generation, application and validation loop

Here, the system starts writing code. The user types an intent ("add a healthcheck route", "rename foo to bar"), and the system produces a patch, checks it, applies it, validates it,

and either commits to the result or rolls back to the exact pre-call state.

The deliverable is a loop, not a list of endpoints. How you expose the steps over HTTP is your call: a single run-everything route, or one route per stage (generate, apply, run the full thing), or something else, as long as a peer can drive a full iteration from the dashboard's *Patch Loop* tab. What is non-negotiable is the behavior.

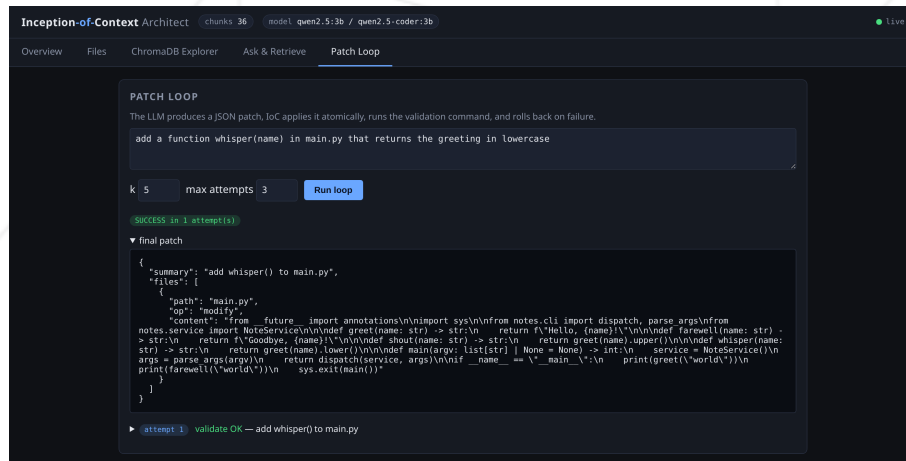


Figure VI.4: *

Patch Loop tab. The intent goes in, the system records each attempt (generation, sanity check, application, validation), and surfaces the final patch as JSON.

The patch is structured JSON, not free-form text and never a diff. The reference shape is `{summary, files: [{path, op, content}]}`, where `op` is one of `create`, `modify`, or `delete`, and `content` holds the complete post-change file. Per-file replacement keeps the apply step deterministic.

The validation command is configurable through an `ioc.config.yml` file at the target project root. On the reference Python stack, `python -m py_compile {files}` over the modified files is a sane baseline; any compile or test command in your target language works the same way.

Before any disk write, the patch has to pass a small set of sanity checks. These are hard refusals, not warnings. Reject a patch that:

- leaks retrieval markers into its content,
- tries to `create` a file that already exists,
- replaces a non-empty file with `""`, `"None"`, or `"null"`,
- defines a function whose body is only a stub (`pass`, `...`, `return None` in Python, or the equivalent in your language),
- would shrink an existing file by more than 60%,
- touches more than three files at once.

Inception-of-Context (IoC)

Once the sanity step is happy, apply atomically, run the validation command, and feed any error log back to the model for another attempt. After **three iterations** without a green validation, give up and roll back.



Rollback must bring the project back to the exact state it was in before the loop started. A partial rollback does not pass this part. Snapshot every touched file before writing.



Before applying, write each modification to a temporary file named *.ioc.tmp and rename it to its final target only when every temporary is ready. `os.replace` is atomic on the same volume.

Chapter VII

Bonus part



The bonus part is graded only if the mandatory part is **perfect**. Perfect means the mandatory part has been fully completed and works without malfunctioning, even when the user pushes the system in weird directions. If you have not passed all the mandatory requirements, your bonus part will not be evaluated at all.

A few ideas, in no particular order:

- `POST /reindex` for an on-demand full reindex. Useful when the watcher misses an event.
- Automatic Git commit of a validated patch, with a commit message generated by the LLM from the patch summary.
- Docker SDK integration: watch a target service's logs and trigger the patch loop on crash.
- A dry-run mode that produces a diff without touching the disk.
- A richer dashboard, with per-chunk relevance score, a visual diff of the proposed patch, and live validation status through SSE.

Chapter VIII

Submission and peer-evaluation

Submit your work on your assigned `Git` repository. Only the code inside that repository will be graded during the defense. As usual, double-check that the official submission is the one being evaluated.

Your repository must contain, at its root, the folders `p1`, `p2`, and `p3`. A `bonus` folder may be added if the mandatory part is fully complete.

A `README.md` at the root has to describe how to start the system, and a `Makefile` at the root has to expose the entry points that bring it up and tear it down, for example `make up` backed by `docker compose up`. The peer must be able to run the project without reading the source.



Never commit a downloaded LLM weight, an embedding model archive, or a populated ChromaDB directory. These files are large, machine-specific, and useless to a reviewer. They belong in a `.gitignore`, not in the repository. A submission that pushes them is rejected on sight.